

SIMATS ENGINEERING
ASSESSMENT TOOL 2: Case Study

COURSE NAME:	Cloud Computing and Big Data Analytics	COURSE CODE:	CSA15 04
---------------------	---	---------------------	-------------

COURSE OUTCOME COVERED

CO3: *Implement cloud-based solutions using cloud storage, web APIs, and platform services for real-world applications. (BL3)*

Dave's Taxonomy: Precision (Level 3)
Question Count: 3

Total Marks: 30

Code of Conduct

I V.RAJ KUMAR(192525208) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____

Assessment Tasks

Case Study

1. Case Study 1: Smart Retail Inventory Management System

A retail chain implements a cloud-based inventory system connected to store devices. Clients (POS systems and dashboards) communicate via APIs to centralized cloud services. Cloud storage stores product data, transaction logs, and analytics datasets. The system uses platform services for analytics and demand forecasting. Secure access is maintained through VPNs, firewalls, and API gateways. This solution integrates infrastructure, services, and web applications for efficient operations.

2. Case Study 2: Cloud-Based Document Collaboration System

An enterprise deploys a document collaboration tool similar to Google Docs. Users access the platform via browsers, enabling real-time editing and sharing. Documents are stored in distributed cloud storage with version control mechanisms. Web APIs enable synchronization across multiple clients and devices.

Network infrastructure ensures low latency and high availability globally. Security includes encryption, identity management, and access permissions for collaboration.

3. Case Study 3: Online Food Ordering Platform

A startup builds a cloud-based food ordering system accessible via web browsers and mobile clients.

Frontend clients interact with backend services through RESTful Web APIs hosted on a cloud platform.

Cloud storage is used to store menu images, invoices, and customer data securely. The infrastructure uses virtual machines and managed Kubernetes for scalability and reliability. Security is enforced

through authentication tokens, HTTPS, and role-based access control. The system demonstrates integration of clients, APIs, storage, and platform services in real time.

Case Study Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Case	25%	Clearly identifies all technical and business requirements	Identifies most requirements	Basic understanding only	Poor understanding of case
Application of Concepts	25%	Applies suitable cloud concepts and tools effectively	Good application with minor issues	Partial application	Weak or incorrect application
Analysis	20%	Strong analysis with clear trade-off reasoning	Reasonable analysis	Limited analysis	Minimal analysis
Solution Design	20%	Solution is feasible, practical, and well justified	Reasonable solution with minor gaps	Basic solution	Unclear solution
Clarity	10%	Well-structured and easy to follow	Mostly clear	Somewhat unclear	Poorly organized

CASE STUDY 1: Smart Retail Inventory Management System

1. Understanding of the Case: Requirements Analysis

1.1 Technical Requirements

- Real-time or near-real-time synchronization of stock levels across hundreds of store locations.
- Durable, queryable storage for structured product catalogues, transactional logs, and analytical datasets.
- A stateless, versioned Web API layer exposing inventory, sales, and forecasting operations to POS and dashboard clients.
- A platform-managed analytics/ML service for demand forecasting without operating custom training infrastructure.
- Network-level isolation between store devices and cloud services (VPN/private connectivity), plus perimeter controls (firewall, API gateway).
- High availability and horizontal scalability to withstand peak shopping events (e.g., festive sales).

1.2 Business Requirements

- Minimize stock-outs and overstocking to protect revenue and reduce holding cost.
- Provide store managers and head-office analysts with a single, trustworthy source of truth for inventory.
- Keep operating cost proportional to actual usage (pay-as-you-go) rather than fixed capital expenditure.
- Ensure regulatory and PCI-adjacent compliance for transaction data, since POS data is sensitive.
- Support rapid onboarding of new stores with minimal on-site IT effort.

2. Application of Concepts: System Architecture

The architecture is presented using AWS services as the primary reference implementation, with GCP/Azure equivalents noted for portability.

Layer	AWS Service	Azure / GCP Equivalent	Purpose
Client connectivity	Site-to-Site VPN / AWS Direct Connect	Azure VPN Gateway / Cloud VPN	Secure private tunnel from store network to cloud VPC
API layer	Amazon API Gateway	Azure API Management / Apigee	Central entry point; throttling, auth, request validation
Compute (transactional)	ECS Fargate / AWS Lambda	Azure Container Apps / Cloud Run	Stateless inventory & order microservices
Operational data store	Amazon DynamoDB	Cosmos DB / Firestore	Low-latency key-value store for live stock counts

2.1 Data Flow

1. A sale or stock adjustment at a POS terminal is sent, over the store's VPN tunnel, to Amazon API Gateway.
2. API Gateway authenticates the request (IAM/Cognito token), applies throttling, and routes it to a microservice on ECS Fargate.
3. The microservice writes the authoritative stock delta to DynamoDB (single-digit-millisecond read/write) and simultaneously emits an event to a Kinesis stream.

3. Analysis and Trade-off Reasoning

Decision Point	Option A	Option B	Recommendation & Rationale
Inventory consistency model	Strong consistency (synchronous writes to a single region)	Eventual consistency (async replication, cache-backed reads)	Eventual consistency for dashboards, strong consistency for the POS write path — DynamoDB supports both per-request, so cost and latency stay low without risking oversell.
Forecasting approach	Managed platform service (SageMaker Forecast)	Custom-built ML pipeline (self-managed training/inference)	Managed service: faster time-to-value, no ML-ops overhead, though it trades away fine-grained model tuning available in a custom pipeline.

4. Solution Design: End-to-End Architecture and Security

4.1 Layered Design

- **Client Layer:** POS terminals and manager dashboards, communicating exclusively over TLS 1.2+ within an IPsec VPN tunnel.
- **Edge/Security Layer:** API Gateway + WAF enforce request validation, rate limiting, and IP allow-listing per store.
- **Application Layer:** Stateless microservices (Inventory, Orders, Forecast-Reader) deployed on ECS Fargate behind an internal load balancer.

4.2 Security Design

- **Network isolation:** Store-to-cloud traffic routed only through VPN; VPC subnets segregate public-facing gateway resources from private data-tier resources.
- **Identity & access control:** Role-Based Access Control (RBAC) via IAM: store staff roles can only mutate their own store's inventory; head-office analysts get read-only cross-store access.

CASE STUDY 2: Cloud-Based Document Collaboration System

1. Understanding of the Case: Requirements Analysis

1.1 Technical Requirements

- Real-time concurrent editing with automatic conflict resolution, targeting sub-100ms perceived edit latency for co-located users.
- Distributed, versioned document storage supporting rollback to any prior revision.
- Web APIs for document CRUD operations, sharing/permission management, and session/presence signalling.
- Multi-region deployment to keep read/write latency low for globally distributed users.
- End-to-end and at-rest encryption with fine-grained identity and access management.

1.2 Business Requirements

- Deliver a collaboration experience competitive with market-leading tools to support enterprise productivity.
- Guarantee document durability and auditability (who changed what, and when) for compliance.
- Support elastic growth in user base and document volume without re-architecture.
- Protect confidential enterprise documents from unauthorized access or leakage.

2. Application of Concepts: System Architecture

Layer	AWS Service	Azure / GCP Equivalent	Purpose
Client transport	API Gateway WebSocket APIs	Azure Web PubSub / Firebase Realtime	Persistent bidirectional channel for live edits and presence
Global entry point	Amazon CloudFront + Global Accelerator	Azure Front Door / Cloud CDN	Routes users to the nearest regional edge for low latency
Real-time sync engine	ECS/EKS service running a CRDT library (e.g., Yjs/Automerger)	Same pattern on AKS/GKE	Merges concurrent edits without central locking

2.1 Data Flow

- A user opens a document; the client establishes a persistent WebSocket connection to the nearest regional real-time sync node via API Gateway.
- Keystrokes are encoded as CRDT operations and broadcast to all connected collaborators through the sync service, achieving convergence without a central lock.

3. Analysis and Trade-off Reasoning

Decision Point	Option A	Option B	Recommendation & Rationale
CAP posture for metadata	Strong consistency (CP)	High availability, eventual consistency (AP)	AP is chosen: collaborators tolerate a few seconds of metadata replication lag far better than an editor becoming unavailable during a network partition.
Concurrency algorithm	Operational Transformation (OT)	Conflict-free Replicated Data Types (CRDTs)	CRDTs recommended: no central transformation server is required, simplifying multi-region scaling, at the cost of

Decision Point	Option A	Option B	Recommendation & Rationale
			slightly larger metadata overhead per operation.
4. Solution Design: End-to-End Architecture and Security			
4.1 Layered Design			
• Client Layer: Browser-based editor maintaining a local CRDT replica for instant local echo of keystrokes. • Edge Layer: CloudFront/Global Accelerator directs each user to the nearest regional cluster. • Real-Time Layer: CRDT-based sync service on EKS, horizontally scaled per document-session shard. • Storage Layer: S3 for immutable delta blocks; DynamoDB Global Tables for cross-region version metadata. • API Layer: REST/GraphQL APIs for document lifecycle, sharing, and search, independent of the real-time channel.			
4.2 Security & Version Control			
• Encryption: Client-side (end-to-end) encryption of document content before it leaves the browser where enterprise policy requires it; server-side KMS encryption otherwise. • Identity management: Azure AD/IAM federation with the enterprise SSO; every API call authorized against document-level ACLs.			

CASE STUDY 3: Online Food Ordering Platform

1. Understanding of the Case: Requirements Analysis

1.1 Technical Requirements

- Elastic scaling to absorb sharp peak-hour demand spikes (lunch/dinner rushes) without over-provisioning at all times.
- Real-time order status tracking pushed to customer and delivery-partner clients.
- Low-latency, reliable delivery of menu images and static assets globally.
- Secure, token-based authentication across web and mobile clients.
- Reliable, ACID-compliant handling of orders and payments alongside flexible catalogue data.

1.2 Business Requirements

- Maintain a smooth ordering experience during peak load to avoid cart abandonment and lost revenue.
- Keep infrastructure cost variable and aligned with actual order volume, appropriate for a startup's budget.
- Build customer trust through transparent order tracking and account security.
- Enable rapid feature iteration (new restaurants, promotions) without downtime.

2. Application of Concepts: System Architecture

Layer	AWS Service	Azure / GCP Equivalent	Purpose
Client entry	Amazon API Gateway / ALB Ingress	Azure API Management / GKE Ingress	TLS termination, routing, request throttling
Compute / orchestration	Amazon EKS (managed Kubernetes)	AKS / GKE	Runs order, catalogue, payment, and tracking microservices with auto-scaling pods
Relational data store	Amazon RDS (PostgreSQL, Multi-AZ)	Azure SQL / Cloud SQL	ACID-compliant storage for orders, payments, users

2.1 Data Flow

- A customer places an order from the web or mobile client; the request hits API Gateway, which validates the JWT issued by Cognito.
- The request is routed into the EKS cluster to the Order microservice, which writes the order transactionally to RDS PostgreSQL and caches cart/session state in Redis.

3. Analysis and Trade-off Reasoning

Decision Point	Option A	Option B	Recommendation & Rationale
Service architecture	Monolith	Microservices on Kubernetes	Microservices: independent scaling of the order service during peak hours without over-scaling catalogue/auth services; accepted cost is higher operational complexity.
Order data store	NoSQL (e.g., DynamoDB/MongoDB)	Relational SQL (PostgreSQL)	SQL for orders/payments: ACID transactions are essential for financial correctness; NoSQL reserved

Decision Point	Option A	Option B	Recommendation & Rationale
			for less transactional data such as catalogue browsing.

4. Solution Design: End-to-End Architecture and Security

4.1 Layered Design

- **Client Layer:** Web (React/Next.js) and native mobile apps consuming versioned REST APIs.
- **API Gateway Layer:** Centralized ingress for auth, throttling, and routing into the Kubernetes cluster.

4.2 Security & Reliability Strategy

- **Authentication:** OAuth2 with short-lived JWTs issued by Cognito; refresh tokens rotated and stored securely on-device.
- **Transport security:** HTTPS enforced end-to-end, including internal service mesh traffic via mTLS within the cluster.
-
-